COMPARING RAINBOW DQN AND PPO ON FLAPPY BIRD AND SUPER MARIO BROS.

Milan Tóth
milan.toth@uni-ulm.de
Ulm University
Ulm, Germany

ABSTRACT

Rainbow DQN and PPO are usually compared at a fixed number of environment steps, which grants the two methods unequal computation, since the number of steps a method completes in an hour is itself a property of the method. We fix wall-clock time instead: each algorithm is tuned under one search protocol and then given the same budget on a dedicated accelerator, 24 hours on Flappy Bird and 30 on Super Mario Bros., learning from pixels on identical observation and reward streams. On Mario we train on twenty levels of the original game and evaluate on twenty-one that are never trained on, twelve of them from Super Mario Bros.: The Lost Levels. On Flappy Bird PPO ends more than an order of magnitude ahead: its greedy policy survives at least seven hours of game time per evaluation episode against Rainbow's nine minutes, and the two sets of evaluation episodes do not overlap. On Mario the two end up close, at 0.58 against 0.48 of a level cleared, and both are weak: neither reaches the flag once in 630 held-out episodes. Every configuration was run at a single seed, so what is compared are runs rather than algorithms. Our implementationis available at: https://github.com/milannal1m/rainbow-vs-ppo.

Keywords Deep reinforcement learning · Rainbow DQN · Proximal policy optimization · Flappy Bird · Super Mario Bros.

1 Introduction

Rainbow DQN and PPO come from the two main families of deep reinforcement learning, one learning what each action is worth and the other representing behaviour directly. Comparisons between such methods are usually made at a fixed number of environment steps, as in the papers introducing both [9, 22]. That measures how much an algorithm extracts from a given amount of experience, and it is the right question when experience is what costs. It is not the question anyone faces when the constraint is a machine and a deadline. The number of steps a method completes in an hour is itself a property of the method: Rainbow takes a gradient step every fourth environment step and maintains a replay buffer besides, where PPO collects a rollout and reuses it for a few epochs before discarding it. Holding steps equal therefore grants one method more computation than the other.

We therefore fix wall-clock time. Each algorithm is tuned under one search protocol and then given the same budget on a dedicated accelerator, 24 hours on Flappy Bird and 30 on Super Mario Bros., and what it achieves within that is what we report. Both learn from pixels alone,

on identical observation and reward streams, so that nothing separates the two runs on a game except the method itself.

The two games are chosen for what they do not share. Flappy Bird has two actions, dense reward and no end: a competent policy plays until it makes one mistake, and the episode has to be stopped by hand. Mario has twelve actions, a terminal event and levels that differ from one another, which makes generalisation measurable. We train on twenty levels of the original game and evaluate on twenty-one that are never trained on, twelve of them from Super Mario Bros.: The Lost Levels, a game the agent has not seen.

Two findings follow. On Flappy Bird PPO ends more than an order of magnitude ahead, on a task where the value both critics learn is provably unable to separate the two policies, and we set out what we can and cannot attribute that to. On Mario the two end up close despite one being tuned and the other not, and neither generalises at all: no held-out level was completed once in 630 attempts by either agent.

2 Related Work

Both games have served as testbeds for the two families compared here. On Super Mario Bros., Klein [12] trains a Double DQN on raw frames and reports that the compute needed to reach five million steps was itself what limited how far the study could be taken. Zahedi et al. [32] compare DQN against PPO from pixels on the same environment and action set used here, training and evaluating primarily on the first level and testing generalisation on the second, whose underground setting produces a markedly different distribution of observations. They find the variation produced by design and hyperparameter choices within each algorithm to exceed the variation between the two. On Flappy Bird, Chen [5] trains a DQN from pixels across three difficulty settings and observes that further training did not reliably improve play, scores instead fluctuating, and Thurler et al. [27] place PPO beside Q-learning and a neuroevolutionary method on the same game.

Two things distinguish the present study. Where the work above stays within the first two levels of Super Mario Bros., we train on twenty and test generalisation on twenty-one held-out levels that are never trained on, twelve of them from a game the agent has not seen. And where Zahedi et al. [32] compare the two algorithms after training runs of different lengths, we hold compute equal: every run is tuned under one search protocol and then given the same wall-clock budget on a dedicated accelerator, so what is compared is not what each algorithm extracts from a fixed amount of experience but what it delivers for a fixed amount of compute.

3 Reinforcement Learning Methods

This section presents the two reinforcement learning methods compared in this work: Rainbow DQN, which learns what each action is worth and acts greedily on those values, and PPO, which represents the behaviour itself and adjusts it directly. Both are described as published; our implementation choices are collected in Appendix A.

3.1 Rainbow Deep-Q-Networks (DQN)

Mnih et al. [19] introduce Deep Q-Networks (DQN), which combine Q-learning [31] with a neural network as function approximator; for pixel inputs, as here, a convolutional one. At each timestep t the agent observes a state (s_t), takes an action (a_t) and receives a scalar reward (r_t) together with a successor state (s_{t+1}); the network Q_θ approximates the action-value function, the return expected from (s_t, a_t) under optimal continuation,

$$Q^*(s_t, a_t) = \max_\pi \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \,\middle|\, s_t,\, a_t,\, \pi\right], \tag{1}$$

where π is the policy followed after a_t, that is the rule by which the agent picks an action in each state it encounters, and the discount factor $\gamma \in [0, 1)$ weights each reward by how far in the future it arrives. Instead of waiting until the episode has ended, when that sum has

been observed, and using it as a label, DQN regresses onto a *bootstrapped* target: the reward actually received, plus a discounted estimate of what follows, read off at the successor state. Two departures from naive regression make this stable. First, each transition (s_t, a_t, r_t, s_{t+1}) is stored in a *replay buffer* $\mathcal{D}$ and sampled uniformly rather than used in the order it was experienced, since consecutive transitions are strongly correlated and drawing them out of order lowers the variance of each update, while a stored transition can be drawn many times instead of being discarded after one [19]. Second, the bootstrap estimate is taken not from the network being trained but from a separate *target network* Q_{θ^-}, a copy of it refreshed only every C steps: an update raising $Q_\theta(s_t, a_t)$ typically also raises $Q_\theta(s_{t+1}, a)$, and hence the very target the prediction is chasing. Mnih et al. [18] introduce the frozen copy to delay that feedback and report that it makes divergence considerably less likely. Target and objective are then

$$y_t = r_t + \gamma \max_a Q_{\theta^-}(s_{t+1}, a), \tag{2}$$

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[\left(y_t - Q_\theta(s_t, a_t) \right)^2 \right]. \tag{3}$$

Actions are chosen ε-greedily during training: uniformly at random with probability ε, and $\arg\max_a Q_\theta(s_t, a)$ otherwise. This balances *exploration*, gathering information about actions whose value is still uncertain, against *exploitation*, taking the action currently believed best [25]; annealing ε downwards shifts that balance towards the latter as the estimates improve. The two policies involved differ, which is what makes Q-learning *off-policy* [31, 25]: the $\max$ in the target evaluates the greedy *target policy* regardless of what the agent actually did next, so learning is not tied to the *behaviour policy* that produced the experience. Here that behaviour policy explores ε-greedily, and older entries in $\mathcal{D}$ come from policies the agent has long since outgrown; neither invalidates the update, which is what licenses the replay buffer at all.

Hessel et al. [9] combine six independent extensions of this algorithm into a single agent, which they call Rainbow (DQN). Their central empirical claim is that the extensions address largely orthogonal weaknesses and therefore compose: the combined agent outperforms every individual variant, and their ablation identifies prioritized replay and multi-step returns as the components whose removal costs the most. We implement all six and enable all six in every Rainbow configuration reported here.

Double Q-learning. The $\max$ operator in the DQN target uses the same network to both select and evaluate the bootstrap action, which systematically overestimates action values under approximation error and noise [26, 28]. Van Hasselt et al. [29] carry the correction proposed by Van Hasselt [28] over to DQN, letting the online network choose the action and the target network evaluate it,

$$y_t = r_t + \gamma \, Q_{\theta^-}\left(s_{t+1}, \, \arg\max_a Q_\theta(s_{t+1}, a) \right). \tag{4}$$

Dueling architecture. In many states the choice of action is irrelevant, so estimating a separate Q value per action wastes capacity. Wang et al. [30] split the network head into a state-value stream $V(s_t)$ and an advantage stream $A(s_t, a_t)$ and recombine them as

$$Q(s_t, a_t) = V(s_t) + A(s_t, a_t) - \frac{1}{|\mathcal{A}|} \sum_a A(s_t, a), \tag{5}$$

where subtracting the mean makes the advantage stream measure how much better an action is than the average action in that state. Since $V(s_t)$ enters $Q(s_t, a)$ for every a, an update on any single action also moves it, so the value of a state is learned from every visit rather than only from the visits on which that particular action happened to be chosen.

Prioritized replay. Sampling the buffer uniformly revisits a transition as often as it happened to occur, which bears no relation to how much the network still stands to learn from it. Schaul et al. [20] estimate that quantity by the magnitude of the transition's last temporal-difference error, $\delta_i = y_i - Q_\theta(s_i, a_i)$, the quantity whose square the loss minimises, and draw transition i with probability $P(i) \propto p_i^\alpha$ from the priority $p_i = |\delta_i| + \epsilon$; the exponent α interpolates between

uniform and fully proportional sampling and ϵ keeps transitions with zero error reachable. New transitions are inserted at maximal priority. Because non-uniform sampling changes the distribution the updates are averaged over, and hence the solution they converge to, each sample is weighted by $w_i = (N \cdot P(i))^{-\beta}$, normalised by $\max_j w_j$ so that no weight exceeds one, with β annealed linearly to 1 over training. We use the proportional variant with a sum tree.

Multi-step learning. The target above commits to a single observed reward before falling back on the network's own estimate. It can instead be extended over n steps [24],

$$y_t = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_a Q_{\theta^-}(s_{t+n}, a), \tag{6}$$

where the bootstrap term is discounted by γ^n rather than γ. Each additional real reward reduces how much of the target rests on an estimate, at the cost of more sampled noise within it, and lets a newly observed reward reach the states preceding it in fewer updates. Hessel et al. [9] find n to be among the more sensitive of Rainbow's hyperparameters, settling on $n = 3$.

Distributional RL. Where standard DQN learns the expected return, a single number per state-action pair, Bellemare et al. [3] learn the full distribution of returns that pair can produce, represented as a categorical distribution over a fixed support of N_{atoms} points $z_j = v_{\min} + j\,\Delta z$ spanning $[v_{\min}, v_{\max}]$, with spacing $\Delta z = (v_{\max} - v_{\min})/(N_{\text{atoms}} - 1)$. The network is the one from DQN with $N_{\text{atoms}} \times N_{\text{actions}}$ outputs instead, a softmax over each action's atoms normalising its distribution separately [9]. Forming the target the same way as before, reward plus discounted continuation, shifts and contracts this support and moves it off the fixed grid, so the result is clipped to $[v_{\min}, v_{\max}]$ and projected back onto it before the cross-entropy between target and prediction is minimised. Bellemare et al. [3] contract by a single γ; in Rainbow, where the distributional loss is combined with multi-step returns, each z_j becomes $r_t + \gamma^n z_j$ [9]. Action selection uses the expectation $Q(s_t, a_t) = \sum_j p_j(s_t, a_t)\, z_j$, leaving the surrounding algorithm unchanged. Returns outside the support cannot be represented, which Hessel et al. [9] observe leads to systematic underestimation when the range is set too narrow.

Noisy Nets. Perturbing actions uniformly at random scales badly when a reward is reachable only through a long and particular sequence of moves. Fortunato et al. [8] therefore move the stochasticity from the action into the weights, adding a parametric noise term to each fully connected layer,

$$y = (b + Wx) + (b_{\text{noisy}} \odot \varepsilon_b + (W_{\text{noisy}} \odot \varepsilon_w)x)\,, \tag{7}$$

with ε_b and ε_w drawn at random and $\odot$ the element-wise product. Since b_{noisy} and W_{noisy} are parameters, gradient descent shrinks them where the agent has settled on an answer and leaves them large where it has not, so exploration becomes state-dependent and decays on its own schedule rather than a prescribed one. Hessel et al. [9] draw the noise in factorised Gaussian form, which needs far fewer random draws per layer, and act fully greedily ($\varepsilon = 0$) wherever noisy layers are used.

3.2 Proximal Policy Optimization (PPO)

Schulman et al. [22] introduce Proximal Policy Optimization (PPO), a policy gradient method. Where DQN learns action values and reads a policy off them, policy gradient methods parametrise the policy itself as $\pi_\theta(a_t \mid s_t)$, a probability distribution over actions, and improve it by gradient ascent on the expected return. The gradient is estimated as

$$\hat{g} = \hat{\mathbb{E}}_t\Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t\Big]\,, \tag{8}$$

where $\hat{\mathbb{E}}_t$ averages over the finite set of transitions gathered before an update and the advantage $\hat{A}_t$ estimates how much better a_t turned out than what the policy would ordinarily have done in s_t. Its sign fixes the direction of the update: actions that beat that baseline have their probability raised, those that fall short have it lowered. Because the expectation is taken under π_θ itself,

the estimate is only valid for the policy that produced the samples, which makes the method *on-policy*.

Taking more than one gradient step per batch is tempting, since collecting the batch was the expensive part, but Schulman et al. [22] observe that it drives the policy far enough from the one that generated the samples for the updates to overshoot badly. Trust Region Policy Optimization (TRPO) addresses this by maximising a surrogate objective subject to a hard constraint on the KL divergence between the old and new policy [23], which requires second-order machinery and rules out sharing weights between policy and value function [22]. PPO seeks the same restraint using first-order optimisation alone.

Let $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ be the probability ratio between the current and the sampling policy, so that $r_t(\theta_{\text{old}}) = 1$. Maximising $\hat{\mathbb{E}}_t[r_t(\theta)\hat{A}_t]$ on its own rewards ever larger departures from θ_{old}, so Schulman et al. [22] bound it by clipping,

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\Big)\Big]. \tag{9}$$

Once r_t leaves $[1-\epsilon, 1+\epsilon]$ the clipped term stops paying for moving it further, and the minimum makes the whole expression pessimistic: a ratio that has drifted counts against the objective, whereas the same drift is disregarded when it would flatter it. Updates are therefore free to move near θ_{old} and gain nothing by straying, without any constraint being stated explicitly.

The advantage is not observed and must itself be estimated, which requires a second learned quantity alongside the policy: a value function $V(s_t)$ predicting the return expected from s_t onwards. This makes PPO an actor-critic method [22]. The policy is the *actor*, which chooses actions; the value function is the *critic*, which judges them but never acts. The two can be entirely separate networks, or two output heads on a shared trunk, as they are here.

The critic enters only through the advantage estimate. Its basic unit is the temporal-difference residual $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$, which compares what one step actually produced, the reward plus the critic's valuation of where it led, against the critic's valuation of where it started; a positive δ_t means the step turned out better than the critic predicted. Generalized Advantage Estimation (GAE) accumulates these residuals over a trajectory segment of length T [21, 22],

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t-1}\delta_{T-1}, \tag{10}$$

a geometrically decaying sum in which λ sets how many steps ahead still carry weight. At $\lambda = 0$ only δ_t survives, which has low variance but is biased whenever V is imperfect; at $\lambda = 1$ the sum reduces to the observed returns minus $V(s_t)$, unbiased but accumulating the noise of every sampled reward along the way. Intermediate values trade the two against each other, and Schulman et al. [21] find the best results at λ close to but below one. Relying on a learned V rather than on returns observed through to the end of an episode is what makes the gradient estimate usable at all, trading a little accuracy for far less noise.

The quantity actually optimised combines three terms [22],

$$L^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t\big[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)\big], \tag{11}$$

where $L_t^{VF} = (V_\theta(s_t) - V_t^{targ})^2$ is a squared-error loss on the value head, regressed onto $V_t^{targ} = \hat{A}_t + V_{\theta_{old}}(s_t)$, the λ-return implied by Equation 10 [21] and the target PPO implementations form [10], and

$$S[\pi_\theta](s_t) = -\sum_a \pi_\theta(a \mid s_t)\log\pi_\theta(a \mid s_t) \tag{12}$$

the entropy of the distribution the actor already emits, so the bonus needs no quantity beyond the policy itself. It is maximised, and is highest when the policy is undecided between actions. The term is not part of PPO's derivation but is inherited from Asynchronous Advantage Actor-Critic (A3C) [17]: since the policy is sampled from rather than acted on greedily, entropy is the only mechanism keeping that distribution from collapsing onto one action, and c_2 controls how strongly it is preserved. Sharing parameters is not without cost, as the two objectives can pull the trunk in different directions; Andrychowicz et al. [2] find separate networks competitive or better in several settings.

Training alternates between collecting and optimising. A fixed number of timesteps is gathered under $\pi_{\theta_{\text{old}}}$, advantages are computed over the whole segment, and the objective is then optimised for several epochs of minibatch stochastic gradient ascent before $\theta_{\text{old}} \leftarrow \theta$ and the batch is discarded [22]. This is where the on-policy character becomes concrete: the ratio $r_t(\theta)$ is only meaningful relative to the policy that drew the actions, so experience cannot be retained beyond the update it was collected for. Nothing corresponding to a replay buffer exists, and the multiple epochs are precisely what the clipping makes safe. How much performance depends on the details surrounding this loop is itself a documented finding [6, 10]; the choices we adopt are listed in Appendix A.

3.3 Relating the two methods

The two algorithms belong to separate families, and Hessel et al. [9] note that their own study stays within the value-based one, leaving policy-based and actor-critic methods aside. Set side by side, they turn out to answer several of the same questions in different ways.

Both are model-free and learn from pixels alone. Both rely on a learned value estimate, though to different ends: for Rainbow it is the object of learning, from which behaviour follows, while for PPO it is auxiliary, its only purpose being to tell the policy whether an action beat expectations. Both bootstrap, and both expose the same dial in the process: how far into the future the target relies on rewards that were actually observed before handing over to the network's own estimate. Rainbow sets it with n, the number of real rewards carried in the target, PPO with λ. Turned one way, either method leans almost entirely on its own predictions and accepts whatever bias they carry; turned the other, it leans on observed outcomes and accepts their noise. Both also need a device to stop an update from destroying what has been learned, Rainbow by freezing the target it regresses towards and PPO by refusing to pay for large changes to the probability ratio, and in both, exploration must be supplied deliberately rather than emerging on its own.

The differences begin with what the network represents. Rainbow outputs a value per action and derives its policy by taking the maximum, so the policy has no parameters of its own; PPO carries it explicitly as a distribution and moves it by gradient ascent.

From this follows the sharpest practical divergence. Because Rainbow's target evaluates the greedy policy whatever the data actually did, transitions stay valid long after the policy that produced them has been superseded and can be replayed many times, whereas PPO's probability ratio is defined against the policy that drew the actions, so a rollout is exhausted within a few epochs and discarded. One method accumulates a large memory and revisits it; the other holds only what it has just collected, with consequences running from memory footprint to what a fixed budget of environment steps buys.

4 Experimental Apparatus

4.1 Environments

Both games are played from pixels. Neither environment is used exactly as distributed: each required corrections to defects that would otherwise have made the observation or the reward signal unreliable. The preprocessing applied identically to both is deferred to Section 4.2.

4.1.1 Flappy Bird

We use `flappy-bird-gymnasium` 0.4.0 [13]. The player controls a bird in continuous descent and must pass through gaps in oncoming pipes; the action space is Discrete(2), idle or flap, and a single mistimed input ends the episode. The environment exposes a 12-dimensional feature vector describing the bird and the nearby pipes, but we discard it and take the rendered frame instead, since the object of this work is learning from raw visual input. The background image is disabled at construction, leaving a uniform fill that we mask to zero.

Rewards are emitted per step: $+1$ for passing a pipe, $+0.1$ for surviving a step on which no pipe is passed, -0.5 for leaving the top of the screen and -1 on collision. The last two overwrite

rather than add to the first two, so a step that both passes a pipe and leaves the screen yields -0.5.

An episode ends only on collision. The environment registers no step limit and its internal score limit is unset, so a competent policy produces an episode of unbounded length; we therefore stop an episode once its cumulative reward reaches 10^5, with consequences for the results taken up in Section 4.5.

The only stochastic element is the vertical placement of each pipe gap, drawn from eight discrete heights using the environment's seeded generator, so an episode is determined by its reset seed. This holds only once three variables the upstream implementation leaks across episodes are reset by hand: the wing-animation cycle, the scroll offset of the ground, and the flap flag. All three are mutated during play, none is reinitialised on reset, and all three reach the rendered frame. Since that frame is our observation, the same seed produced different pixel sequences depending on how long the preceding episode had run. We correct this with a wrapper applied directly above the environment.

4.1.2 Super Mario Bros.

We use `gym-super-mario-bros` 9.1.0 on `nes-py` 9.0.1 [11], which exposes the original NES game through an emulator at 240×256 RGB. Actions are the COMPLEX_MOVEMENT set of 12 button combinations: movement left or right, each alone or combined with jump, run or both, together with jump on its own, up, down and a no-op. The run button doubles as the fire button once a fire flower has been collected, and crouching is what enters a pipe, so the set covers the game's interactions without a dedicated item control.

The reward is dominated by progress: the increase in the player's horizontal position beyond the furthest point already reached, paid per frame and capped at five pixels, so that moving right earns reward continuously while moving left or standing still earns nothing. To this the environment adds a small penalty proportional to elapsed game time, -25 on death, $+50$ on reaching the flag, and minor terms for score, coins and power-ups. Each frame's composed reward is then clipped by the environment to its own declared range of $[-15, +15]$. This binds on the terminal events: a death frame reaches the caller as -15 rather than -25, and a flagpole frame as $+15$ rather than $+50$, the stated magnitudes surviving only as a diagnostic in the info dictionary.

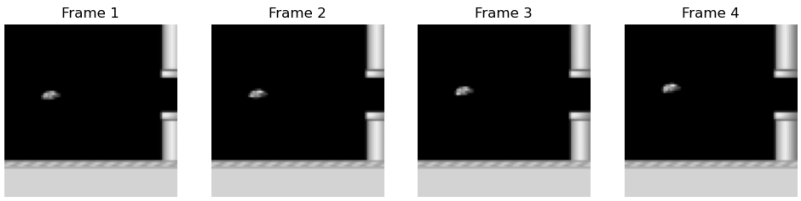
Two defects in that signal required correction, both concerning the position from which progress is computed. It is read from two memory bytes that underflow when the player walks left past the level origin, wrapping to a large value; because the environment latches its high-water mark before checking whether the implied progress is plausible, a single underflow suppresses all further progress reward for the rest of the episode. We detect implausible values, by absolute bound and by frame-to-frame jump, and restore the mark. Separately, the position counter restarts whenever the player enters a pipe, vine or warp while the high-water mark does not, so progress after an area transition is measured against a mark belonging to a different area. We track one mark per area and restore the appropriate one on transition, which also prevents terrain being paid for twice on return from a bonus room.

Each episode is played on a level drawn from a configured set, selected by a hash of the reset seed so that the assignment is reproducible. Training uses twenty levels of the original game, and a further twenty-one are held out and never trained on, twelve of them drawn from *Super Mario Bros.: The Lost Levels*. Maze levels, whose non-monotonic layout makes horizontal progress a misleading reward, are excluded throughout, and the partition is checked at import for overlap.

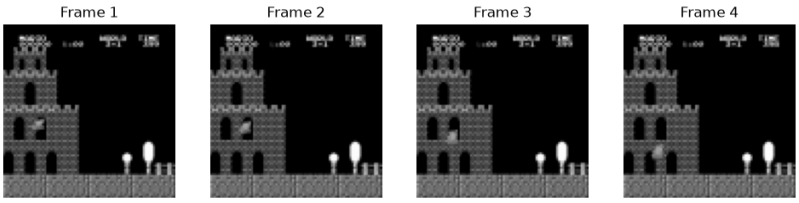
Episodes terminate on death or on reaching the flag, and are truncated after 3000 agent steps. In practice the in-game clock expires earlier, at roughly 2000 steps, which kills the player and so surfaces as a termination.

The emulator is deterministic, which allows a policy to succeed by memorising a sequence of inputs rather than by responding to what it sees. We follow Machado et al. [16], who introduce sticky actions against the same determinism in the Arcade Learning Environment:

with probability 0.25 the previously executed action is repeated instead of the one selected. Each episode additionally begins with a random number of up to 30 no-op steps [18]. Both are active during evaluation as well as training.



(a) Flappy Bird, $4 \times 80 \times 80$.



(b) Super Mario Bros., $4 \times 84 \times 84$.

Figure 1: One observation as the network receives it, for each game.

4.2 Preprocessing

What reaches the network is not what the environment emits. Frames are reduced in size and colour and given a short history, and Mario's reward is aggregated and rescaled. Both transformations are built from the environment configuration alone, so the observation and reward stream a learner sees is fixed by the choice of game and is identical whichever algorithm is trained on it.

From frame to tensor. Each rendered frame is resized before it is converted to grayscale, so that the interpolation runs on the colour image. Flappy Bird is reduced to 80×80, the resolution used by Lin [15], and Super Mario Bros. to the 84×84 of the Atari benchmark [18]. Four consecutive frames are stacked along the channel axis, giving the network the short history it needs to infer velocity from static images [18], and the result is scaled to $[0, 1]$.

Mario is played at a coarser granularity than the emulator runs: each selected action is held for four emulator frames, so one agent step covers four frames of game time, and the observation for that step is the element-wise maximum of the last two of them [18]. Both are conventions carried over from the Atari benchmark, where the repetition buys throughput at little cost in control and the maximum compensates for objects the console draws only on alternate frames. The tensor reaching the network is therefore $(4, 80, 80)$ or $(4, 84, 84)$ in single precision.

Rewards. Flappy Bird is trained on the reward the environment emits, unchanged. Mario's is summed over the four frames spanned by one agent step, and that sum Σ_t is clipped to $[-15, +15]$ and divided by 15, so an ordinary step pays in $[-1, +1]$. Summing rather than averaging is the Atari convention and leaves the return of the coarse-grained process equal to the environment's own [18, 16]; the constant is the range the environment itself enforces per frame (Section 4.1.2), so a scaled reward of 1 is one maximal frame reward as the environment defines it.

To this we add one deviation of our own. Since the environment has already clipped the completion bonus to $+15$ (Section 4.1.2), the transform above pays $+1.000$ for reaching the flag against the $+0.800$ of sustained running, which is to say that it asks the agent to travel right rather than to finish. This is the cost Mnih et al. [19] accept when they introduce clipping: rewards of different size become indistinguishable to the agent. We therefore exempt the

completion component from the clip, reading it back as $b_t \in \{0, 50\}$ from the info dictionary and adding it in the same unit,

$$r_t = \frac{\mathrm{clip}(\Sigma_t,\, -15,\, +15)}{15} + \frac{b_t}{15}, \tag{13}$$

so that every ordinary step is unchanged and a completed level pays $+4.33$. The exemption is ours, not the environment's.

Neither algorithm applies any further transformation of its own. In particular PPO is run without the observation and reward normalisation that is otherwise standard for it, since a normaliser fitted to its own running statistics would break exactly the equality described above; the point is taken up in Appendix A.

4.3 Architecture

Figure 2 gives both networks. The two agents are trained separately and share no parameters; what they share is the specification of the convolutional trunk, whose convolutions are those of Mnih et al. [18]: an 8×8 convolution with stride 4 into 32 channels, a 4×4 convolution with stride 2 into 64 channels and a 3×3 convolution with stride 1 into 64 channels, each followed by a rectifier. Where Mnih et al. [18] rely on strides alone to reduce resolution and Lin [15] pool after every convolution, we pool once, after the first, and pad the three convolutions by 2, 1 and 1 where Mnih et al. [18] pad none. The padding is what brings both input sizes to the same $5 \times 5 \times 64$, so both games arrive at 1600 features.

Only the heads differ, and they differ because the two methods predict different things. PPO's actor emits one logit per action, which a softmax turns into the policy $\pi_\theta(a_t \mid s_t)$ it samples from, and its critic emits the single number $V(s_t)$. Rainbow emits neither a policy nor a scalar value but a distribution: its value stream produces N_{atoms} logits and its advantage stream $|\mathcal{A}| \times N_{\mathrm{atoms}}$, the two are combined per action by Equation 5, and a softmax across the atoms of each action normalises its return distribution separately [9]. Acting requires collapsing that distribution back to a scalar, as described. The width H shared by all layers above the trunk is a hyperparameter of both methods; the value selected for each configuration is given in Appendix C.5.

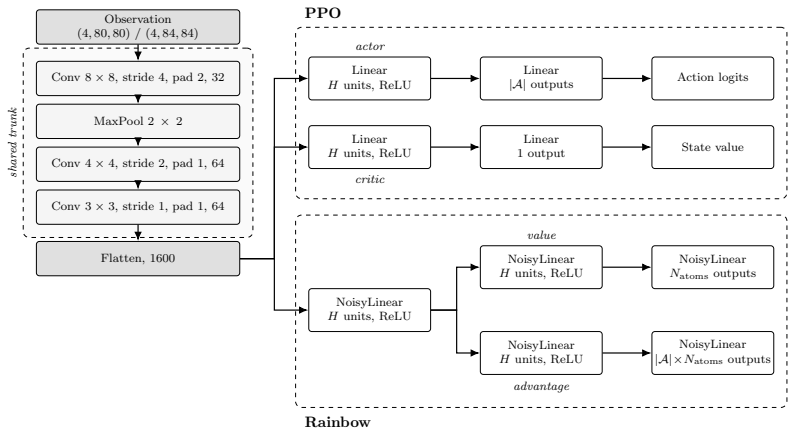


Figure 2: The PPO and Rainbow architectures implemented in this work. H is the hidden width, searched over for both methods (Appendix C.5).

Table 1 gives the resulting sizes, and they do not order the two methods consistently. On Mario, where the search gave both a width of 512, Rainbow is roughly two and a half times the size of PPO; on Flappy Bird, where it selected 256 for Rainbow and 512 for PPO, Rainbow is the smaller of the two. Part of the difference is therefore an outcome of the search rather than a property of either method.

Table 1: Trainable parameters per configuration. All use a hidden width of 512 except Flappy Bird Rainbow, for which the search selected 256 (Appendix C.5). Rainbow figures are for the online network; the target network is a second copy of the same size.

Game	Agent	Actions	Atoms	Parameters
Flappy Bird	Rainbow	2	51	1 239 506
Flappy Bird	PPO	2	—	1 718 947
Super Mario Bros.	Rainbow	12	101	4 115 170
Super Mario Bros.	PPO	12	—	1 724 077

The rest follows from what each head has to represent, and is best read at a fixed width. There the wide layer above the trunk costs the two exactly the same, PPO holding two copies of it at one parameter per weight and Rainbow one noisy copy at two. What Rainbow adds is the pair of stream layers its shared layer feeds into, and an advantage stream that grows with the atom count as well as the action count: at twelve actions and 101 atoms that layer is some twelve times what two actions and 51 atoms would buy, where PPO's actor gains roughly five thousand parameters over the same change. Equalising the counts would mean departing from the form in which one method or the other was published.

The trunks are identical in specification but not in initialisation. PPO initialises orthogonally with layer-dependent gain, $\sqrt{2}$ through the trunk and hidden layers, 0.01 at the actor output so that the initial policy is close to uniform, and 1.0 at the critic output [10]; Rainbow uses the framework defaults, as Hessel et al. [9] prescribe no scheme of its own. This is the one respect in which the two do not begin from the same place. It and the other small departures from the published algorithms, on which PPO in particular relies, are set out in Appendix A.

4.4 Training

Both agents interact with a single environment instance; neither runs vectorised copies. Episodes are seeded consecutively, so the sequence of pipe placements or Mario levels a run encounters depends only on how many episodes it has played.

Training Loop. PPO alternates between collecting and optimising, following the procedure set out in Section 3.2. A rollout of a fixed number of steps is gathered under the current policy, each action drawn from the categorical distribution the actor's logits define, so an action is taken with the probability the policy assigns it; this is where PPO's exploration comes from, and the entropy term of Equation 11 is what keeps that distribution from collapsing onto one action. Advantages are computed across the whole segment, and the objective is then optimised for several epochs of minibatch gradient ascent, each epoch drawing a fresh permutation of the rollout without replacement. A rollout of N steps optimised for E epochs in minibatches of M transitions yields EN/M gradient updates, so PPO performs one update per M/E environment steps. Rollout length, epoch count and minibatch size are all hyperparameters subject to the search described in Section 4.6. The learning rate is set once per iteration as a linear function of the environment steps taken so far, decaying to a floor and holding there, a schedule Huang et al. [10] report as standard practice rather than one prescribed by Schulman et al. [22].

Rainbow instead updates continuously once the replay buffer has accumulated 20 000 transitions, matching the warmup of Hessel et al. [9], and thereafter performs a gradient update after every fourth environment step as they do. Each update samples a prioritised minibatch, takes a single optimiser step and writes the resulting errors back as priorities. The target network is refreshed by copying the online weights outright at a fixed step interval rather than by a running average, as in Mnih et al. [18].

Rainbow therefore performs one gradient update per four environment steps against PPO's one per M/E, which the selected configurations put at one per 64 on Flappy Bird and one per 16 on Mario (Appendix C.5). The difference in throughput is not attributable to that schedule alone:

the buffer write and the sum-tree maintenance are paid on every step irrespective of whether an update follows, and on Mario Rainbow carries the larger network as well.

Hardware. All runs were executed on bwUniCluster 3.0, on nodes carrying NVIDIA A100 80 GB PCIe accelerators. Every run had one such accelerator to itself; no two runs shared a device, so wall-clock time is comparable across them. Host memory and CPU cores were requested per run at whatever level that run needed, generously enough that neither was ever the limiting factor. This matters because the two methods do not consume host memory alike: Rainbow's replay buffer holds hundreds of thousands of stacked frames and its resident footprint exceeds PPO's by an order of magnitude. Since neither allocation was ever binding, that difference does not enter what is being compared.

Training budget. Comparisons of this kind are usually made at a fixed number of environment steps, as in both papers whose methods we compare [9, 22], which measures how much an algorithm extracts from a given amount of experience. We fix wall-clock time instead. Every run is given a fixed wall-clock budget on the hardware above, 24 hours on Flappy Bird and 30 on Mario, and whatever a method manages within that is what is reported. The question is therefore not which algorithm learns more per transition, but which delivers more for a given compute budget.

The two are not interchangeable, because the number of steps a method completes in an hour is itself a property of the method, as the update schedules above make plain. Equalising that difference away would hide a cost that is real for anyone running these methods.

Seeds and checkpointing. Every configuration was run once, at a single seed. At 24 to 30 hours per run, repeating each configuration across seeds was beyond the compute available, so what follows is a comparison of single runs, without variance estimates or interval statistics. Reported numbers come not from the final weights but from the best checkpoint of a run, selected by a greedy probe episode taken every 1000 training episodes.

4.5 Metrics

We report episode reward as the environment emits it, before the transformation of Section 4.2 that the agent sees, so that the figures remain comparable to other work on these environments. Reward is shaped in both games, so one task-level quantity is reported beside it: pipes passed on Flappy Bird, and on Mario how far through the level the agent got. Episode length is given in seconds rather than steps, since the two games advance at 30 and 15 agent steps per second of game time. Every episode is logged with these, along with the environment steps and wall-clock seconds elapsed, so any curve can be plotted against either axis; curves are smoothed by a trailing mean over 100 episodes.

Evaluation is greedy: PPO takes the mode of its action distribution rather than sampling, and Rainbow's noisy layers act on their mean weights with the noise suppressed.

Flappy Bird is evaluated over 25 episodes, reported as the mean and standard deviation of reward, pipes passed and episode length. Since an episode ends only on collision, a policy that never collides is stopped by the cap of Section 4.1 rather than by the game, and wherever that cap binds, reward and length measure the cap and not the policy.

Mario, unlike Flappy Bird, has a completion event. A run that never learns to reach the flag scores zero however far it travelled, so progress is reported beside the flag rate: the number of pages cleared as a fraction of the level's length, one page being the 256 pixels of a single NES screen. Normalising matters because the levels are not of a kind. They run from eight pages to twenty-three, so an absolute count averaged over a set of them is dominated by the longest. Level lengths come from a route table built once and frozen, measured from completed episodes where possible and parsed from the ROM object list otherwise.

4.6 Hyperparameter Optimization

Both algorithms are tuned with Optuna [1], using its tree-structured Parzen estimator [4] in the multivariate form, which models the searched parameters jointly rather than one at a time. Trials are pruned by Hyperband [14]: a trial whose intermediate score falls behind its cohort at a rung is stopped rather than run out. Each trial uses a single seed, and each study is bounded by a wall-clock budget and a cap on the number of trials, whichever binds first.

A trial is a shortened proxy of the run it is meant to predict. Mario needs the longer proxy because a trial trains on all twenty training levels, so the per-level budget is what binds; Flappy Bird is a single task with dense reward and separates earlier. Each trial is scored by a greedy evaluation of the policy it ends with. On Flappy Bird we take the 25th percentile of episode return, since evaluation episodes there are effectively unbounded and a single long one dominates the mean; the percentile selects configurations that are reliably good rather than occasionally lucky. On Mario that statistic collapses to near zero for almost every configuration, because flag rates across twenty levels are mostly zero, so the mean is used instead. The objective therefore differs between games, but within a game both algorithms are scored on the same statistic, the same task and the same protocol. On Mario, trials train and are scored on the twenty training levels alone; no held-out level enters pruning, selection or tuning at any point.

Table 2: The four searches. Proxy horizons are in environment steps, where M $= 10^6$. Trials started are what the estimator saw, since a pruned trial still carries an intermediate score and rules its region out; trials completed are those evaluated at the full proxy horizon, and so the ones whose scores are directly comparable.

Study	Proxy horizon	Budget	Started	Completed	Ended by
Flappy Bird PPO	1 M	200 / 24 h	200	73	trial cap
Flappy Bird Rainbow	1 M	200 / 24 h	175	55	wall clock
Mario PPO	3 M	100 / 48 h	95	49	wall clock
Mario Rainbow	3 M	100 / 48 h	13	0	failure

Nine parameters are searched for PPO and eight for Rainbow, over ranges identical in both games. Appendix C lists them together with the excluded parameters and the reason for excluding each. Both PPO searches settled within 0.2 % of the same learning rate and on the same epoch count despite running independently on different games, which is some evidence that they recovered a signal rather than noise.

Mario Rainbow is the exception in Table 2 and we report it as such. All thirteen of its trials failed with out-of-memory errors caused by a misconfiguration on our part, and the compute available did not allow the study to be repeated. The configuration we train with is therefore the best of those thirteen, chosen on the intermediate scores they had reached when they died, none of which was measured at the full proxy horizon. It is close to a configuration drawn at random rather than the product of a search, and it is not comparable to the tuning Mario PPO received. Whatever the two achieve on Mario must be read against that asymmetry, and we return to it in Section 7.

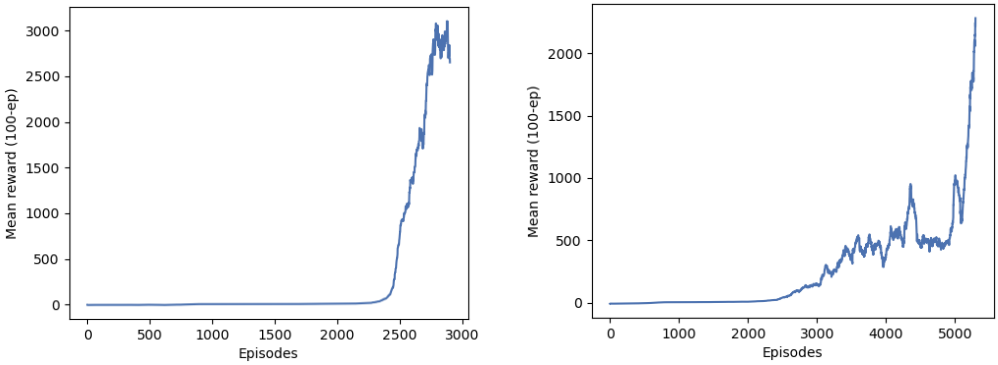
All four studies predate the exemption of Section 4.2 and ran under the plain transform, in which a completed level paid $+1.000$ against the $+0.800$ of sustained running. The exemption changes exactly one step type, which occurs at most once per episode and did not occur in the Mario trials at all, since no trial policy reached a flag; every step the searches observed is unchanged. The configurations are nonetheless optimal for a signal in which completion carried no distinctive weight, and were not re-selected under the corrected one.

The configuration each of the four studies produced, in the sense just described, is given in Appendix C.5.

5 Results

5.1 Flappy Bird Results

Figure 3 gives the reward each run reached over training, as a trailing mean over 100 episodes. Both curves are of the exploring policy: PPO samples from its distribution and Rainbow's noisy layers are active throughout, so neither is the greedy behaviour evaluated below. Rainbow rises sharply at around 2 400 episodes and levels off near 3 000, holding there with visible oscillation over its final few hundred episodes. PPO leaves the floor at a similar point but climbs gradually, sits near 500 for close to two thousand episodes, and then rises steeply to roughly 2 300 at the moment the budget expires. Rainbow's run had settled; PPO's was still rising when the clock ran out.



(a) Rainbow (b) PPO

Figure 3: Reward over training on Flappy Bird, as a trailing mean over 100 episodes. Note the differing axes: Rainbow plays roughly 2 900 episodes in its 24 hours and PPO roughly 5 300.

PPO also plays more episodes, roughly 5 300 against Rainbow's 2 900. The update schedules of Section 4.4 account for the throughput: Rainbow takes a gradient step every four environment steps where PPO takes one every 64 (Appendix C.5), and pays the replay write and sum-tree maintenance on every step besides. Episode length works against this, since both runs play longer episodes as they improve.

Both agents were then evaluated greedily over 25 episodes as described in Section 4.5, at the best checkpoint of each run rather than at its final weights. Table 3 gives the result, and it is one-sided: PPO outperforms Rainbow on every quantity reported, by more than an order of magnitude on each.

Table 3: Greedy evaluation on Flappy Bird, over 25 episodes at the best checkpoint of each run. Values are the mean and standard deviation across episodes, with reward as the environment emits it. PPO's figures are lower bounds: 21 of its 25 episodes were stopped by the reward cap of Section 4.1.1 rather than by a collision, so what its mean and spread measure there is the cap and not the policy.

	Rainbow	PPO
Reward	$2\,053 \pm 1\,853$	$\geq 93\,860 \pm 16\,681$
Best episode	7 237	100 000 (cap)
Pipes passed	439 ± 397	$\geq 20\,112 \pm 3\,575$
Episode length (s)	553 ± 499	$\geq 25\,253 \pm 4\,488$
Episode length (h:mm:ss)	0:09:13	$\geq$ 7:00:53
Episodes stopped by the cap	0 / 25	21 / 25

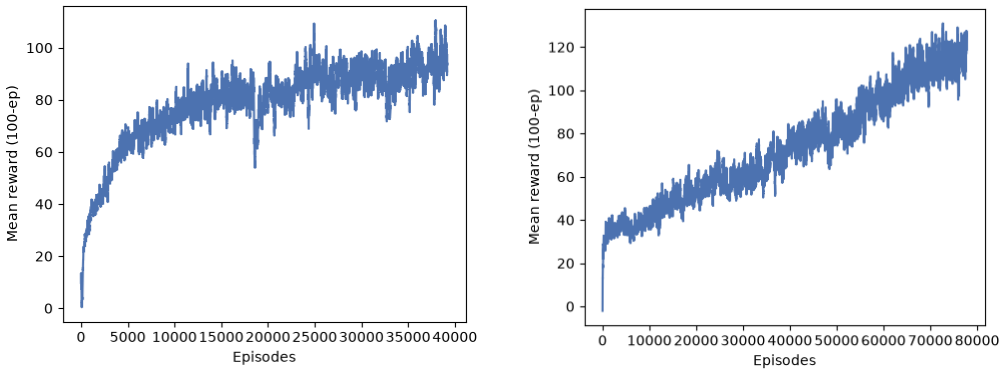
The gap is wide enough that the two sets of episodes do not overlap at all. Rainbow's best of 25 episodes passes 1 550 pipes, while 21 of PPO's 25 run long enough to reach the cumulative

reward of 100 000 at which Section 4.1.1 stops an episode, and are ended there rather than by a collision, at over 20 000 pipes apiece. PPO's figures are therefore lower bounds throughout: its mean episode lasts at least seven hours of game time, and what the remaining spread measures is how far the four uncapped episodes fell short of the cap, not how much the policy varies. How far short they fall is indicated by a single episode we ran without the cap, which collided after roughly eighteen hours of game time at a reward of 241 810, some two and a half times what the cap allows. Rainbow's episodes end on collision without exception, and its spread is its own.

Acting greedily is not worth the same to both. Rainbow's greedy policy scores below the 3 000 its exploring policy reached in training, where PPO's scores more than forty times the 2 300 of its own. The difference between the two runs is therefore made at the transition to greedy acting rather than during training, and we take it up in Section 6.

5.2 Super Mario Bros. Results

Figure 4 gives the reward each run reached over training, as a trailing mean over 100 episodes, again of the exploring policy and, for Rainbow, of the untuned configuration of Section 4.6. PPO plays roughly 78 000 episodes against Rainbow's 39 000, largely from the update schedules of Section 4.4, though the factor between the two is not the factor between their update rates, since an agent that survives longer plays fewer episodes per hour. Rainbow reaches roughly 85 within its first ten thousand episodes and drifts only slightly higher, to about 100, over the remaining thirty. PPO climbs throughout and ends near 120 with no sign of flattening.



(a) Rainbow (b) PPO

Figure 4: Reward over training on Super Mario Bros., as a trailing mean over 100 episodes. Each episode is played on one of the twenty training levels, drawn by a hash of its seed, so the trailing mean averages over levels of differing difficulty as well as over time. Note the differing axes: Rainbow plays roughly 39 000 episodes in its 30 hours and PPO roughly 78 000.

The wide band on both curves is not instability. Each episode is played on one of the twenty training levels, drawn by a hash of its seed (Section 4.1.2), so a trailing mean over 100 episodes averages over a changing mixture of tasks that differ in length and in the skill they demand, and so in the reward they pay.

Both agents were then evaluated greedily as described in Section 4.5, over 30 episodes on each of the 41 levels, at the best checkpoint of each run. Table 4 gives the aggregated results, and Appendix D the results for each level individually. Neither agent learned to finish levels reliably. On the twenty training levels Rainbow reaches the flag in 9% of episodes and PPO in 12%, and both get about half way through a level before dying, PPO the further of the two at 0.58 against 0.48. PPO is ahead on both quantities, but by less than the levels differ among themselves, so the gap is small relative to the spread it is read against.

The two also spread that success differently. Rainbow reaches the flag at least once on seven of the twenty training levels and PPO on only five, four of them shared, so PPO's higher rate comes from finishing fewer levels more reliably.

Table 4: Greedy evaluation on Super Mario Bros., 30 episodes on each of the 41 levels. Flag is the fraction reaching the flagpole and progress the fraction of the level cleared, both averaged over the levels of a block, with the deviation across levels rather than episodes.

		Rainbow		PPO	
Levels	n	Flag	Progress	Flag	Progress
Training	20	0.09 ± 0.19	0.48 ± 0.23	0.12 ± 0.28	0.58 ± 0.27
Held-out					
Super Mario Bros.	9	0.00 ± 0.00	0.14 ± 0.11	0.00 ± 0.00	0.17 ± 0.14
The Lost Levels	12	0.00 ± 0.00	0.14 ± 0.06	0.00 ± 0.00	0.08 ± 0.03
All	21	0.00 ± 0.00	0.14 ± 0.08	0.00 ± 0.00	0.12 ± 0.10

On the twenty-one held-out levels neither agent reached the flag once in 630 episodes, and progress falls to 0.14 for Rainbow and 0.12 for PPO, roughly a seventh of a level. Two levels are cleared to no pages at all by either agent, and no held-out level is taken past half. What separates the two agents is where the collapse is worst. PPO holds up on the nine held-out levels of the original game, at 0.17 against Rainbow's 0.14, but falls to 0.08 on the twelve Lost Levels where Rainbow holds 0.14. The advantage PPO carries on familiar levels does not survive the change of game.

Table 5 gives how those episodes ended. Pits account for 33% of Rainbow's training deaths and 21% of PPO's, and for 49% and 62% respectively on the held-out levels. A pit has to be jumped at the right moment, and the agents evidently know where the pits of the training levels are in a way they do not on levels they have not seen.

Table 5: How episodes ended, as a percentage of the 600 episodes on the training levels and the 630 on the held-out ones.

	Levels	Enemy	Pit	Timeout	Flag
Rainbow	Training	51.3	33.3	6.3	9.0
	Held-out	51.3	48.7	0.0	0.0
PPO	Training	46.8	21.0	20.3	11.8
	Held-out	36.8	61.6	1.6	0.0

The timeout column separates the two agents. On the training levels PPO runs the clock out in 20% of episodes against Rainbow's 6%, which matches what its videos show: the agent halts in front of an obstacle, or runs into it repeatedly without trying anything else, sometimes within sight of the flag. Level 1-1 is the clearest case. PPO clears 8.9 of its 12 pages but ends all 30 episodes on the clock, at 5 distinct positions, so it is not dying at varied points but arriving at a handful of places and staying there. On the held-out levels timeouts all but disappear for both, at 2% and 0%, which is not an improvement. An episode has to survive around two thousand steps to reach the clock, and neither agent lasts that long on a level it does not know.

6 Discussion

Flappy Bird. As to why acting greedily is worth so much more to PPO than to Rainbow, a single mistimed input ends the episode, so what matters is not how good an action distribution is on average but how often it puts weight on the wrong action, compounded over tens of thousands of steps. Taking the mode of PPO's distribution removes that weight outright, where suppressing Rainbow's noise leaves the arg max as it was except where the two actions were close enough for the noise to reorder them.

Why PPO ends so far ahead is not something this study can settle, and the obvious explanations do not survive contact with the data. Compute is not it: Rainbow performs a gradient update every four environment steps against PPO's one per 64 (Appendix C.5) and still settles first. Nor is either critic at fault. Both converge to the value the task dictates, $\bar{r}/(1-\gamma) = 12.4$: Rainbow's mean Q is 12.3834 against a predicted 12.383, PPO's mean V is 12.3889 against 12.389. What that value cannot do is separate the two policies, since a discounted sum saturates at the reward rate however long the policy survives, so Rainbow's mean episode of $16\,588$ steps and PPO's of at least $757\,595$ are worth the same to their critics. That is a property of the measure rather than an account of the plateau, and Rainbow went on improving well after its Q values had flattened.

Three measurements would settle it, none of which we took. The first is the action gap of Farahmand [7], the margin by which the better action leads: an arg max preserves the correct ordering only where that margin exceeds the error in the estimates, so a narrow gap turns an approximation error into a wrong action. The states to look at are those immediately preceding a death, where a wide gap would rule this out, the value having carried the information that the arg max then failed on. The second is repetition across seeds, without which no difference reported here can be separated from the variation two runs of the same configuration would have shown (Section 7). The third is a longer clock. PPO was still climbing steeply when the budget expired and Rainbow was not, and Rainbow's own curve had been flat for two thousand episodes before it rose sharply at around $2\,400$, so the 499 episodes over which it then held steady are no evidence that it would not have moved again.

Mario. Rainbow's Mario configuration is the best of thirteen trials that all crashed, which is close to a configuration drawn at random, while PPO's came out of an almost completed search of 95 trials. On that footing the two end up close: 0.58 progress against 0.48 on the training levels, by less than the levels differ among themselves, and 0.12 against 0.14 on the held-out ones. Three readings fit. Rainbow may be less sensitive to its hyperparameters than the asymmetry of Section 4.6 would suggest; Mario at this budget may be far enough beyond both agents that tuning has little to bite on; or the thirteen crashed trials may simply have contained a good configuration, since one drawn close to at random is not thereby a bad one. Nothing here separates them.

Generalisation is where both fail alike, and it is the most robust result here precisely because it is negative and symmetric. Neither reached a flag once in 630 held-out episodes, and progress falls from roughly half a level to roughly an eighth. Twelve of those levels come from a game neither agent has seen, but the collapse is already complete on the nine that do not. No seed, no search and no choice of budget accounts for that.

Two things plausibly contribute. The first is that neither agent is far enough along: a policy that clears about half a training level has not yet learned to jump gaps or handle enemies in general, and what it has learned may be closer to the particular obstacles it has seen than to anything that would carry over. Held-out performance would then be low because there is little to transfer, not because transfer failed. The second is that nothing in our procedure asked for generalisation. Trials were scored on the twenty training levels alone (Section 4.6), so a configuration that traded some of that score for behaviour transferring better would have been pruned for it.

7 Limitations

Every configuration was trained once, at one seed, because the wall-clock budget made repetition impossible. No difference reported above therefore carries a variance estimate, and none can be separated from the variation two runs of the same configuration would have shown anyway. What has been compared are runs, not algorithms.

Our hyperparameter search on Mario adds a second problem on top of this. The two algorithms were not tuned to a comparable standard there, for the reasons given in Section 4.6: PPO's configuration came out of an almost completed search, Rainbow's out of a handful of trials that all failed early. Rainbow's Mario numbers are therefore those of an untuned configuration, and where it falls behind PPO, the search and not the algorithm may be what we are measuring.

Tuning also rests on a proxy one to three million steps long, standing in for runs an order of magnitude longer; configurations that win early and configurations that win late need not be the same set.

Fixing compute rather than experience makes the result a statement about these implementations on this hardware. One choice matters most: both agents interact with a single environment instance, where PPO is ordinarily run over many environments in parallel [22, 10]. Collection is a larger share of PPO's step than of Rainbow's, whose gradient batch comes from the replay buffer rather than from the environment, so running one instance plausibly costs PPO more throughput than Rainbow, by an amount we do not measure. PPO is likewise run without observation and reward normalisation, which Huang et al. [10] count among the canonical implementation details, at a cost we equally do not measure.

A third mismatch runs in the same direction: the reported Mario runs use the corrected reward transform of Section 4.2, the searches that selected their hyperparameters do not. Both algorithms train under the identical transform, so the comparison between them is unaffected, but the absolute level of completion-related results on Mario is that of configurations chosen under a signal that did not distinguish reaching the flag from running towards it.

8 Conclusion

We compared Rainbow DQN and PPO from pixels under equal compute rather than equal experience. On Flappy Bird the two are not close: PPO's greedy policy survives at least seven hours of game time per episode against Rainbow's nine minutes, and the two sets of evaluation episodes do not overlap. On Mario they are close, and both are weak. Neither finishes a training level reliably, and on the twenty-one held-out levels neither reaches the flag in 630 episodes, with progress falling from about half a level to an eighth. That failure is the firmest result here, because it is negative, symmetric across the two methods and does not depend on the budget axis we chose.

What the comparison cannot support is a verdict on the algorithms. Every configuration was run once, and Rainbow's Mario configuration came out of a search that failed outright, so what we have compared are runs. Repeating that search is the first thing we would do differently: it is the one asymmetry in the study that was neither intended nor informative. Beyond it, the questions Section 6 leaves open are all measurable. Whether Rainbow's action gap is narrow where it matters can be read off the states preceding a death; whether either result survives is a matter of more seeds; and whether Rainbow's plateau was final needs only a longer clock. None of the three requires a new method, only further experiments.

Acknowledgements

I thank Dr. Sebastian Gottwald for supervision and feedback throughout this project. I acknowledge support by the state of Baden-Württemberg through bwHPC.

GenAI Usage Disclosure

I employed Claude for grammar and spelling correction, rephrasing and structural editing, for writing and debugging the code used in the experiments, and for discussing technical arguments, locating and checking sources, and drafting passages. I reviewed all material, verified it against the cited sources and revised it, and I take full responsibility for the paper.

References

[1] Takuya Akiba et al. "Optuna: A next-generation hyperparameter optimization framework". In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2623–2631.

[2] Marcin Andrychowicz et al. “What matters in on-policy reinforcement learning? a large-scale empirical study”. In: *arXiv preprint arXiv:2006.05990* (2020).

[3] Marc G Bellemare, Will Dabney, and Rémi Munos. “A distributional perspective on reinforcement learning”. In: *International conference on machine learning*. Pmlr. 2017, pp. 449–458.

[4] James Bergstra et al. “Algorithms for hyper-parameter optimization”. In: *Advances in neural information processing systems* 24 (2011).

[5] Kevin Chen. “Deep reinforcement learning for flappy bird”. In: *CS 229 Machine-Learning Final Projects* (2015).

[6] Logan Engstrom et al. “Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO”. In: *CoRR* abs/2005.12729 (2020). arXiv: 2005.12729. URL: https://arxiv.org/abs/2005.12729.

[7] Amir-massoud Farahmand. “Action-gap phenomenon in reinforcement learning”. In: *Advances in neural information processing systems* 24 (2011).

[8] Meire Fortunato et al. “Noisy Networks For Exploration”. In: *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. URL: https://openreview.net/forum?id=rywHCPkAW.

[9] Matteo Hessel et al. “Rainbow: Combining improvements in deep reinforcement learning”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018.

[10] Shengyi Huang et al. “The 37 implementation details of proximal policy optimization”. In: *The ICLR Blog Track 2023* (2022).

[11] Christian Kauten. *Super Mario Bros for OpenAI Gym*. GitHub. 2018. URL: https://github.com/Kautenja/gym-super-mario-bros.

[12] Sean Klein. *Deep Q-Learning to Play Mario*. CS229 Final Project Report, Stanford University. 2016. URL: https://cs229.stanford.edu/proj2016/report/klein-autonomousmariowithdeepreinforcementlearning-report.pdf.

[13] Martin Kubovčík. *flappy-bird-gymnasium: A Gymnasium environment for the Flappy Bird game*. https://github.com/markub3327/flappy-bird-gymnasium. Version 0.4.0. 2023.

[14] Lisha Li et al. “Hyperband: A novel bandit-based approach to hyperparameter optimization”. In: *Journal of machine learning research* 18.185 (2018), pp. 1–52.

[15] Yen-Chen Lin. *Using Deep Q-Network to Learn How to Play Flappy Bird*. GitHub. 2016. URL: https://github.com/yenchenlin/DeepLearningFlappyBird.

[16] Marlos C Machado et al. “Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents”. In: *Journal of Artificial Intelligence Research* 61 (2018), pp. 523–562.

[17] Volodymyr Mnih et al. “Asynchronous Methods for Deep Reinforcement Learning”. In: *Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*. Ed. by Maria-Florina Balcan and Kilian Q. Weinberger. Vol. 48. JMLR Workshop and Conference Proceedings. JMLR.org, 2016, pp. 1928–1937. URL: http://proceedings.mlr.press/v48/mniha16.html.

[18] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. In: *Nature* 518.7540 (2015), pp. 529–533. DOI: 10.1038/NATURE14236. URL: https://doi.org/10.1038/nature14236.

[19] Volodymyr Mnih et al. “Playing atari with deep reinforcement learning”. In: *arXiv preprint arXiv:1312.5602* (2013).

[20] Tom Schaul et al. “Prioritized Experience Replay”. In: *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. 2016. URL: http://arxiv.org/abs/1511.05952.

[21] John Schulman et al. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. 2016. URL: http://arxiv.org/abs/1506.02438.

[22] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).

[23] John Schulman et al. “Trust region policy optimization”. In: *International conference on machine learning*. Pmlr. 2015, pp. 1889–1897.

[24] Richard S Sutton. “Learning to predict by the methods of temporal differences”. In: *Machine learning* 3.1 (1988), pp. 9–44.

[25] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press Cambridge, 1998.

[26] Sebastian Thrun and Anton Schwartz. “Issues in using function approximation for reinforcement learning”. In: *Proceedings of the fourth connectionist models summer school*. Vol. 255. Hillsdale, NJ. 1993, p. 263.

[27] Leonardo Thurler et al. “Ai game agents based on evolutionary search and (deep) reinforcement learning: A practical analysis with flappy bird”. In: *International Conference on Entertainment Computing*. Springer. 2021, pp. 196–208.

[28] Hado Van Hasselt. “Double Q-learning”. In: *Advances in neural information processing systems* 23 (2010).

[29] Hado Van Hasselt, Arthur Guez, and David Silver. “Deep reinforcement learning with double q-learning”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 30. 1. 2016.

[30] Ziyu Wang et al. “Dueling network architectures for deep reinforcement learning”. In: *International conference on machine learning*. PMLR. 2016, pp. 1995–2003.

[31] Christopher JCH Watkins and Peter Dayan. “Q-learning”. In: *Machine learning* 8.3 (1992), pp. 279–292.

[32] Nika Zahedi, Nils Kuhn, and Evelyn Yee. *MARIO: Reinforcement Learning on Image Observations*. 2025.

A Implementation Details of the Methods

Rainbow. All six extensions are enabled in every Rainbow configuration reported here. We depart from Hessel et al. [9] in two respects. Priorities are computed from the absolute temporal-difference error between the expectations of the predicted and target distributions, whereas Hessel et al. [9] prioritise by the Kullback-Leibler loss once distributional learning is enabled, on the grounds that it is what the algorithm minimises and may be more robust in stochastic environments; the expectation-based form keeps a single code path for the prioritised buffer whether or not the distributional component is active. Second, Mario uses 101 atoms against the 51 of Hessel et al. [9], because its return distribution spans a considerably wider support and the atom count is what determines the resolution Δz at which that support is represented. Gradients are also clipped at a global norm of 1, a value Hessel et al. [9] do not specify; Wang et al. [30] use 10.

PPO. The performance PPO is reported to achieve depends substantially on implementation choices that its original description leaves open, to the point that Engstrom et al. [6] attribute much of its advantage over TRPO to them rather than to the clipped objective. We follow the canonical set catalogued by Huang et al. [10]: orthogonal initialisation as described in Section 4.3, an Adam epsilon of 10^{-5}, advantages normalised across the full rollout before minibatching, gradients clipped by global norm, and the value of the final state used to bootstrap a rollout that ends mid-episode. Three further choices from that catalogue are deliberately omitted. Value-loss clipping is not used, following the finding of Engstrom et al. [6] that it does not reliably help. Observation and reward normalisation are not used either, and here the reason is specific to this study: a normaliser fitted to PPO's own running statistics would break the equality of the reward signal that Section 4.2 rests on. Early stopping on a KL threshold is likewise omitted, leaving the clipping as the sole constraint on update size.

Two choices depart from the canonical set. The learning rate is annealed linearly towards a floor of 10^{-5} rather than to zero, because runs here terminate on wall-clock rather than at a predetermined step count, and a schedule targeting zero would either arrive early or be truncated arbitrarily. Second, GAE distinguishes truncation from termination: a rollout cut short by the time limit bootstraps from the value of the state it was cut at, whereas one ending in death or at the flag does not. Treating the two alike would teach the critic that reaching the time limit is worthless, which on Mario is the outcome of most competent episodes.

B The Two Environments Side by Side

Table 6 collects what the two games look like to an agent once the corrections of Section 4.1 and the preprocessing of Section 4.2 have been applied.

C Hyperparameter Search

C.1 Study configuration

Each of the four studies of Table 2 is a single Optuna study backed by file storage and shared by several concurrent workers on one node of four A100 accelerators, twelve for PPO and four for Rainbow, the difference reflecting the memory a replay buffer occupies. Every trial uses the seed of the full runs. Table 7 gives the settings of the sampler and the pruner.

Mario's PPO trials are pruned at a later rung than its Rainbow trials. Two configurations measured across the training split are only 7 % apart at 500 000 steps but 31 % apart at one million, so a rung placed at proxy/9 would have discarded trials that go on to win.

C.2 Searched parameters

Table 8 gives the nine parameters searched for PPO and the eight for Rainbow. Hidden width appears in both, setting the width of the layers above the convolutional trunk described in Section 4.3.

Table 6: The two environments as the agent meets them.

	Flappy Bird	**Super Mario Bros.**
Observation	$(4, 80, 80)$, grayscale	$(4, 84, 84)$, grayscale
Actions	2: flap, no-op	12: left and right, each alone or with jump, run or both; plus jump, up, down, no-op
Frame skip	none	4, observation is the max of the last two
Reward	survival: $+0.1$ per frame; pipe passed: $+1$; off screen: -0.5; collision: -1	progress: pixels moved right per frame, capped at $+5$; clock tick: -1; flag: $+50$; death: -25; minor score terms. Clipped to ± 15 per frame
Reward preprocessing	none	$r_t = \dfrac{\mathrm{clip}(\Sigma_t,\, \pm 15)}{15} + \dfrac{b_t}{15}$ with the completion bonus $b_t \in \{0, 50\}$
Episode ends	collision only	death, flag, or the level timer running out
Stochasticity	pipe gap height (8 possibilities)	sticky actions $p = 0.25$, up to 30 no-op starts
Task metric	pipes passed	flag rate, pages cleared, level progress

Table 7: Sampler and pruner settings. The budgets that bound each study are given in Table 2.

	Flappy Bird	Super Mario Bros.
Uniform startup trials	10	5
First Hyperband rung	proxy$/9$	proxy$/3$ (PPO), proxy$/9$ (Rainbow)
Pruning reduction factor	3	3
Objective	25th percentile	mean
Greedy evaluation episodes per trial	30	300
Training levels per trial	—	20 (training split)

Two PPO bounds were raised after a first study and the studies rerun: the entropy coefficient from 10^{-3} to $5 \cdot 10^{-3}$, since the first study drove it towards its lower bound and collapsed the policy entropy, and the minimum rollout length from 1024 to 4096, short rollouts yielding advantage estimates too noisy to rank configurations by.

C.3 Quantities held constant

Not every parameter of either method is searched. Table 9 lists those held fixed and the reason in each case. Two of the entries are there to protect the comparison rather than to save budget: the replay period, which would otherwise let the tuner decide how much gradient work Rainbow receives per transition, and the discount factor, held at the same value for both so that neither search spans a larger or more consequential set of knobs than the other.

The environment is untouched by the search as well. Frame stacking, observation size, action set, frame skip, reward clipping, sticky actions, no-op starts, episode limit and level split are fixed by Section 4.1 and Section 4.2 and identical between the algorithms, so no configuration can win a trial by being tested on an easier task.

C.4 From proxy to full run

Some parameters are defined relative to the length of the run they belong to, so a trial cannot carry the value the exported run carries, and the configuration that wins a study is not the one that was tested verbatim. The step budget and PPO's learning-rate anneal length are set to the proxy horizon inside a trial and to the full run's length on export. Rainbow's replay capacity is

Table 8: Searched parameters. Ranges are identical in both games; *log* marks a logarithmically sampled range.

Algorithm	Parameter	Range
PPO	Learning rate	$[10^{-5}, 10^{-3}]$, log
	Epochs per rollout	$\{3, 4, 6, 10\}$
	Minibatch size	$\{64, 128, 256\}$
	Rollout length	$\{4096, 8192\}$
	ϵ (clip)	$\{0.1, 0.2, 0.3\}$
	c_2 (entropy)	$[5 \cdot 10^{-3}, 5 \cdot 10^{-2}]$, log
	c_1 (value)	$[0.3, 1.0]$
	λ (GAE)	$[0.9, 0.99]$
	Hidden width	$\{256, 512\}$
Rainbow	Learning rate	$[10^{-5}, 10^{-3}]$, log
	Batch size	$\{32, 64, 128\}$
	Target sync interval	$\{500, 1000, 2000, 8000\}$
	n (multi-step)	$\{1, 3, 5\}$
	α (PER)	$[0.3, 0.7]$
	β_0 (PER)	$[0.3, 0.6]$
	σ_0 (noisy)	$[0.3, 0.7]$
	Hidden width	$\{256, 512\}$

Table 9: Parameters excluded from the search.

Parameter	Value	Reason
γ (both)	0.99	Standard, and held so that the two searches span equally consequential sets
Gradient clip (PPO)	0.5	Standard for PPO, low sensitivity
LR floor (PPO)	10^{-5}	Floor of the schedule, not a free choice
Six Rainbow components	all enabled	They define the algorithm
Replay period (Rainbow)	4	Fixes the compute each transition receives
Adam ϵ (Rainbow)	$1.5 \cdot 10^{-4}$	Value of Hessel et al. [9]; torch's 10^{-8} inflates the effective step for small gradients
N_{atoms}, v_{min}, v_{max}	51, -5, 20 (FB); 101, -15, 100 (Mario)	A property of the return scale, which differs by roughly a factor of six between the games
Gradient clip (Rainbow)	1.0	Not specified by Hessel et al. [9], see Appendix A
ϵ-greedy schedule	inert	Superseded by the noisy layers

rescaled differently, and its β anneal length is set to the proxy horizon in a trial but to a fixed value on export rather than to the run's length, so that β reaches 1 well before a run ends.

Rainbow's replay capacity is set to 100 000 inside a trial rather than to the capacity the exported run carries, so that the buffer holds a comparable fraction of the trial's history. Matching the absolute size instead would have let a trial hold its entire history in memory, a regime the full run never enters. It remains a deviation, since a proxy one to three million steps long cannot reveal what buffer a run an order of magnitude longer wants.

C.5 Selected configurations

Tables 10 and 11 give the configuration each study produced. The Mario Rainbow column is the best of thirteen failed trials rather than the winner of a completed study, for the reasons given in Section 4.6.

Table 10: PPO configurations selected by the search.

Parameter	Flappy Bird	Super Mario Bros.
Learning rate	$2.034 \cdot 10^{-4}$	$2.038 \cdot 10^{-4}$
Epochs per rollout	4	4
Minibatch size	256	64
Rollout length	4096	8192
ϵ (clip)	0.1	0.3
c_2 (entropy)	$1.514 \cdot 10^{-2}$	$1.015 \cdot 10^{-2}$
c_1 (value)	0.721	0.692
λ (GAE)	0.958	0.916
Hidden width	512	512
Resulting update rate	one per 64 steps	one per 16 steps

Table 11: Rainbow configurations selected by the search.

Parameter	Flappy Bird	Super Mario Bros.
Learning rate	$1.325 \cdot 10^{-4}$	$2.978 \cdot 10^{-4}$
Batch size	128	128
Target sync interval	8000	2000
n (multi-step)	3	3
α (PER)	0.620	0.625
β_0 (PER)	0.301	0.400
σ_0 (noisy)	0.425	0.559
Hidden width	256	512
Resulting update rate	one per 4 steps	one per 4 steps

Where the two configurations of a method diverge is largely where the games differ: Mario's PPO takes the wider clip and the longer rollout, and its Rainbow the shorter target-sync interval and the larger initial noise. Read against Section 4.4, the last row of each table is what separates the two methods under a wall-clock budget. Rainbow buys one gradient update with four environment steps in both games, where PPO needs 64 on Flappy Bird and 16 on Mario, so Rainbow spends between four and sixteen times as much computation per transition as PPO does.

D Per-level Mario Results

Table 12: Per-level greedy evaluation on Super Mario Bros., 30 episodes per level. Max is the length of the level in pages and progress the fraction of it cleared. Distinct is the number of distinct terminal positions. Lengths are measured from completed episodes where possible and parsed from the ROM object list otherwise; Lost Levels A-1 could not be measured.

		Rainbow				PPO			
Level	Max	Flag	Pages	Prog.	Dist.	Flag	Pages	Prog.	Dist.
Training levels									
1-1	12	0.10	8.73	0.73	20	0.00	8.93	0.74	5
1-2	10	0.00	3.97	0.40	25	0.07	4.97	0.50	20
1-3	9	0.00	2.00	0.22	13	0.00	2.00	0.22	9
1-4	8	0.47	5.10	0.64	19	0.83	8.00	1.00	17
2-1	12	0.00	5.20	0.43	27	0.00	3.10	0.26	22
2-2	12	0.00	4.33	0.36	24	0.00	3.30	0.27	25
2-3	13	0.00	6.67	0.51	24	0.00	10.83	0.83	19
3-1	12	0.00	4.43	0.37	28	0.00	6.33	0.53	27
3-3	9	0.00	4.70	0.52	26	0.00	7.30	0.81	17
3-4	8	0.50	6.27	0.78	20	0.67	7.80	0.97	20
4-1	13	0.60	12.17	0.94	10	0.03	9.53	0.73	18
5-1	12	0.03	5.23	0.44	27	0.00	2.73	0.23	13
5-4	8	0.07	5.90	0.74	23	0.00	6.07	0.76	22
6-1	11	0.03	8.83	0.80	24	0.77	10.73	0.98	7
6-2	13	0.00	2.83	0.22	19	0.00	5.60	0.43	28
6-4	8	0.00	4.23	0.53	30	0.00	6.37	0.80	26
7-1	11	0.00	1.80	0.16	29	0.00	4.97	0.45	21
7-3	13	0.00	4.40	0.34	22	0.00	6.80	0.52	25
8-1	23	0.00	3.13	0.14	16	0.00	5.60	0.24	15
8-3	13	0.00	4.53	0.35	28	0.00	3.77	0.29	14
Held-out, Super Mario Bros.									
5-3	9	0.00	1.87	0.21	16	0.00	1.93	0.21	13
7-2	12	0.00	2.47	0.21	23	0.00	2.17	0.18	26
2-4	8	0.00	2.97	0.37	24	0.00	3.83	0.48	18
3-2	12	0.00	0.07	0.01	11	0.00	2.97	0.25	14
4-2	13	0.00	0.00	0.00	13	0.00	0.00	0.00	13
4-3	8	0.00	1.00	0.12	13	0.00	1.00	0.12	11
5-2	12	0.00	2.10	0.18	29	0.00	0.57	0.05	18
6-3	10	0.00	1.13	0.11	20	0.00	1.07	0.11	13
8-2	13	0.00	1.23	0.09	24	0.00	1.43	0.11	18
Held-out, The Lost Levels									
LL 1-1	12	0.00	2.00	0.17	23	0.00	0.60	0.05	17
LL 1-3	11	0.00	1.00	0.09	24	0.00	1.00	0.09	7
LL 2-1	15	0.00	2.23	0.15	26	0.00	1.00	0.07	12
LL 3-1	15	0.00	1.97	0.13	22	0.00	1.53	0.10	15
LL 4-1	13	0.00	1.80	0.14	21	0.00	1.10	0.08	4
LL 4-3	12	0.00	0.97	0.08	21	0.00	1.00	0.08	5
LL 5-1	11	0.00	2.00	0.18	27	0.00	1.13	0.10	14
LL 6-1	11	0.00	1.63	0.15	23	0.00	1.10	0.10	16
LL 7-1	11	0.00	2.00	0.18	29	0.00	1.13	0.10	17
LL 8-1	11	0.00	1.93	0.18	28	0.00	1.33	0.12	21
LL C-3	11	0.00	2.10	0.19	27	0.00	1.17	0.11	15
LL A-1	—	0.00	0.00	—	17	0.00	0.00	—	16